An Application of the Implicit Function Theorem

Justin Chiu
Cornell Tech
jtc257@cornell.edu

March 26, 2026

Abstract

Gradient-based learning forms the foundation of modern machine learning, and automatic differentiation allows ML practitioners to easily compute gradients. While automatic differentiation only costs a constant multiple of the time and space required to evaluate a function, it has its limitations. In particular, when evaluating a function itself requires many computations, the direct application of reverse-mode automatic differentiation may quickly run out of memory. In this report, we review the implicit function theorem (IFT) and its use in reducing the cost of computing gradients in scenarios where function evaluation is expensive, focusing on the application of the IFT to differentiating through the solutions of optimization problems.

1 Introduction

Gradient-based learning underpins many of the recent successes in machine learning, particularly advances involving neural networks. The key to the success of gradient-based methods is automatic differentiation (AD), which greatly increases the development speed of machine learning research by allowing practitioners to circumvent the error-prone and time-consuming process of computing gradients manually. AD operates by reducing functions into compositions of atomic operations, for which we have a library of derivatives for, and composing those derivatives via the chain rule. The underlying concept behind AD is that a program's execution trace is a valid and useful representation of a function [Griewank and Walther, 2008], where the execution trace is a history of all the computation performed during the evaluation of a program used to express a function.

Storing the execution trace of a program allows AD systems to easily compute derivatives. However, longer execution traces can quickly consume a large amount of memory. Consider an iterative method, such as gradient descent, whose execution trace takes the form of an unrolled loop: Given an initial point $\theta = x_0 \in \mathbb{R}^n$, iterates $x_1, x_2, ..., x_{K-1}$ and final point $x = x_K$ are produced by running the gradient descent update for K iterations, i.e. $x_{t+1} = x_t - \alpha \nabla f(x_t)$. In order to differentiate through this procedure with reverse-mode AD (i.e. compute $\frac{dx}{d\theta}$), we have to store all the x_k iterates as well the computation used to produce them. Thus, the memory complexity of storing this execution trace scales linearly in the number of iterations K as well as the dimensionality

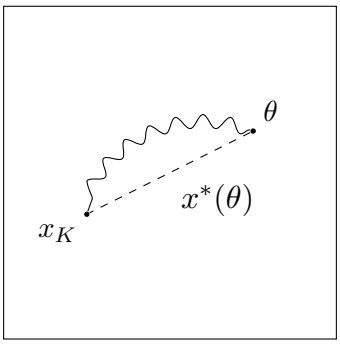

Figure 1: Reverse-mode automatic differentiation of an iterative method consumes memory linear in the number of iterations, illustrated by the squiggly line (the path of the iterates) from initial point θ to the final point x. Such methods are not uncommon, found in bilevel optimization problems such as hyperparameter optimization and metalearning. The implicit function theorem (IFT) provides conditions under which there exists an implicit function, $x^*(\theta) = x$ (dashed line), which we can use to compute the derivative $\frac{dx_K}{d\theta}$ without storing all intermediate computation.

of the iterates $n = \dim(x_k)$[1]. For large K, this can be infeasible as the space complexity is at least $\Omega(Kn)$, as this ignores the traces produced by the updates themselves. One method for overcoming the space complexity's dependence on the number of iterations K in the above example is to use the implicit function theorem (IFT), letting you throw away intermediate computation (e.g. iterates x_1 through x_{K-1}) while still being able to compute the derivative $\frac{dx}{d\theta}$. Naively, if the Jacobian of the local system is dense, this comes at the cost of $O(n^2)$ space and $O(n^3)$ time, trading off the dependence on K for a larger dependence on n.[2] We provide an illustration in Fig. 1.

Large execution traces are not uncommon; optimization problems are often solved with iterative methods, resulting in traces very similar to the example given above. In order to speed up solvers for these problems, one could learn an initialization by differentiating through the execution trace of the iterative method [Finn et al., 2017, Kim et al., 2018, Venkataraman and Amos, 2021]. In this report, we will cover the use of the IFT as a method for dealing with the space complexity of AD in exactly these cases. In particular, we will focus on applying the IFT to differentiating the solutions to optimization problems [Amos and Kolter, 2017, Agrawal et al., 2019]. We will walk through an example by applying the IFT to an optimization problem that is equivalent to softmax, then show how the approach is generalized [Amos and Kolter, 2017].

[1]Forward-mode would get away from the linear dependence on K, but would require $n = \dim(\theta)$ sweeps. If all sweeps are performed in parallel, forward-mode AD would take $\Omega(n^2)$ memory as opposed to $\Omega(Kn)$ for reverse-mode.

[2]The $O(n^2)$ space comes from storing a dense $n \times n$ Jacobian (or a factorization thereof), and the $O(n^3)$ time comes from solving the associated dense linear system. This will become clear when we cover the IFT. This can be reduced by using approximate inverse-Jacobian or inverse-Hessian products, as we will show later in this report.

2 Related Work

There are a variety of methods for reducing the space limitations of AD, of which we only mention three: checkpointing, reversible computation, and implicit differentiation.

The first method, checkpointing, improves space complexity at the cost of time [Griewank and Walther, 2008]. Rather than storing the full execution trace of a program, checkpointing instead recomputes values when needed. This can result in a slowdown due to recomputation, and also requires careful choosing of which part of the trace to checkpoint and recompute.

A second method is reversible computation [Maclaurin et al., 2015, Gomez et al., 2017], which improves space complexity at the cost of expressivity, but not speed. Reversible computation ensures that a function's derivative depends only on the output, allowing the input to be discarded during function evaluation. This is typically accomplished by ensuring that the input is easily reconstructed from the output, restricting the expressivity of layers.

A third method uses the IFT, which we focus on. Application of the IFT potentially improves space complexity at the cost of stronger assumptions. The IFT gives conditions under which derivatives can be computed independent of intermediate computation, with the primary condition being the characterization of the output as the solution to a system of equations.

One of the main applications where the space complexity of AD limits its scalability is bilevel optimization. Bilevel optimization problems are optimization problems with another nested inner optimization problem embedded within.

The application we focus on in this report is expressing individual layers of a neural network declaratively as the solution of an optimization problem [Amos and Kolter, 2017, Agrawal et al., 2019, Gould et al., 2019]. This allows models to learn, without heavy manual specification, the constraints of the problem in addition to the parameters of the objective. An example of this is learning to play Sudoku from only input-output pairs [Amos and Kolter, 2017].

Other applications that can be formulated as bilevel optimization problems are hyperparameter optimization, metalearning, and variational inference. Hyperparameter optimization formulates hyperparameter tuning as a bilevel optimization problem, as for each hyperparameter configuration a new model must be trained as the inner loop [Maclaurin et al., 2015, Lorraine et al., 2019b,a, Bertrand et al., 2020]. Derivatives must then be propagated through the inner training loop to the outer hyperparameter loop. Similarly, metalearning learns the parameters of a model such that the model is able to quickly be adapted to a new task via gradient descent [Finn et al., 2017, Rajeswaran et al., 2019]. This is accomplished by differentiating through the learning procedure of each new task. Finally, a variant of variational inference follows a very similar format: semi-amortized variational inference (SAVI) aims to learn a model that is able to provide a good initialization for variational parameters that are subsequently updated iteratively to maximize a lower bound objective [Kim et al., 2018]. This is also accomplished by differentiating through the iterative optimization procedure applied to the variational parameters during inference.

In all the above applications, the inner-loop optimization problem is solved with an iterative method, except in rare, simple cases. The IFT reduces the memory footprint of automatic differentiation, which would otherwise be difficult to scale.

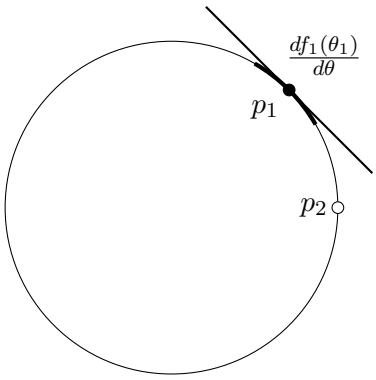

Figure 2: A circle, defined by the relation $F(\theta, x) = \theta^2 + x^2 - 1 = 0$. We view θ as a parameter, x as a solution to $F(\theta, x) = 0$ given θ and the pair (θ, x) is a solution point. Our goal is to compute the derivative $\frac{dx}{d\theta}$. While we cannot differentiate the relation F directly as it is not a function, we can compute the derivative at the solution point $p_1 = (\theta_1, x_1)$ using the local parameterization (or solution mapping) $f_1(\theta) = \sqrt{1 - \theta^2}$. This parameterization holds in a neighbourhood around p_1, visualized as an arc. We cannot use the same parameterization at $p_2 = (\theta_2, 0)$ as the derivative is undefined.

3 The Implicit Function Theorem

The implicit function theorem (IFT) has a long history, as well as many applications in a wide variety of fields such as economics and differential geometry. For an overview of the history of the IFT and some of its classical applications in mathematics and economics, see the book by Krantz and Parks [2003].

Given a relation and some technical conditions that we will go over soon, the IFT is used to identify the existence of local function parameterizations for particular points, and compute the Jacobians of the local function parameterizations for points where the conditions for the IFT hold. As a motivating example, consider the unit circle, governed by the relation $F(\theta, x) = \theta^2 + x^2 - 1 = 0$, which can be interpreted as a system of equations. We would like to compute the derivative $\frac{dx}{d\theta}$. As F fails the vertical line test, we cannot write x as a function of θ globally and therefore cannot compute a derivative that holds globally. However, we can use local parameterizations: $f_1(\theta) = \sqrt{1 - \theta^2}$ on the upper semicircle ($x > 0$) or $f_2(\theta) = -\sqrt{1 - \theta^2}$ on the lower semicircle ($x < 0$). Note that the local parameterizations are functions that hold only within a neighbourhood of a particular solution point (θ, x); there may be solution points where we simply cannot compute a particular derivative, e.g. $\frac{dx}{d\theta}$ when $x = 0$. These local parameterizations then allow us compute the derivative $\frac{dx}{d\theta}$ locally at particular solution points (θ, x) using the corresponding parameterization. When the conditions of the IFT do not hold a particular point, there may still exist a local parameterization and derivative; however, we can neither use the IFT to guarantee the existence of the parameterization nor compute the derivative. See Fig. 2 for an illustration.[3] The IFT generalizes this example, and formalizes the conditions under which there exist continuous local parameterizations for a given relation or system

[3] We will see that the conditions of the IFT do not hold at $p_2 = (\theta_2, 0)$, since the derivative is undefined at that point.

of equations.

While the unit circle in this example has very simple local parameterizations, in general local parameterizations can be more complicated.[4] Additionally, the IFT does not give the form of the local parameterizations; it only guarantees the existence of one around a point and a way to compute its derivative. The local parameterization is left implicit, hence the 'implicit' in IFT.

Formally, given a system of equations $F(\theta, x) = \mathbf{0}_m$,[5] and a solution point $(\theta, x) \in \mathbb{R}^n \times \mathbb{R}^m$, the IFT gives sufficient conditions under which x can locally be written as a function of just the parameters θ within a neighbourhood of the solution point (θ, x). We saw this in the unit circle example, where the local parameterizations were valid around a particular solution point. We refer to this function $x^*(\theta) = x$ as a solution mapping, x a solution, and θ as parameters. These conditions are as follows:

1. We have a solution point (θ, x) that satisfies the system of equations $F(\theta, x) = 0$.
2. F has at least continuous first derivatives: $F \in \mathcal{C}^1$, the class of continuously differentiable functions.
3. The Jacobian matrix of F wrt x evaluated at the solution point (θ, x) is nonsingular:

$$\det \frac{dF(\theta, x)}{dx} \neq 0.$$

Assuming these conditions hold for F at (θ, x), the IFT asserts the existence of the implicit solution mapping $x^*(\theta) : \mathbb{R}^n \to \mathbb{R}^m$ (at the solution point (θ, x)), and that the derivative of the solution mapping wrt the parameters is given by

$$\frac{dx^*(\theta)}{d\theta} = -\left[\frac{dF(\theta, x)}{dx}\right]^{-1} \frac{dF(\theta, x)}{d\theta} \in \mathbb{R}^{m \times n}. \tag{1}$$

As long as we have chosen a solution point (θ, x) that satisfies the conditions, we can ignore how we arrived at that solution point and still compute the derivative of the solution mapping with the IFT. It is important to note that the IFT is local and applies to particular solution points. This means that we can pick and choose which solution points we restrict our analysis to.

Notation: Jacobian Matrix Given a function $F : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^m$, we denote the Jacobian matrix evaluated at the point $(\theta, x) \in \mathbb{R}^n \times \mathbb{R}^m$ as

$$\frac{dF(\theta, x)}{d(\theta, x)} = \begin{bmatrix} \frac{dF(\theta,x)}{d\theta} & \frac{dF(\theta,x)}{dx} \end{bmatrix},$$

[4] Recall that a program trace is a valid representation of a function. One could have a local parameterization that is the trace of a long program, such as one that consists of a series of iterative updates.

[5] We denote vectors and matrices of all 1s and 0s by $\mathbf{1}_S$ and $\mathbf{0}_S$, where S denotes the shape, i.e. $\mathbf{0}_m \in \mathbb{R}^m$.

where we have the matrix of partial derivatives

$$\frac{dF(\theta, x)}{d\theta} = \begin{bmatrix} \frac{dF_1(\theta,x)}{d\theta_1} & \cdots & \frac{dF_1(\theta,x)}{d\theta_n} \\ \vdots & \ddots & \vdots \\ \frac{dF_m(\theta,x)}{d\theta_1} & \cdots & \frac{dF_m(\theta,x)}{d\theta_n} \end{bmatrix} \in \mathbb{R}^{m \times n},$$

and similarly for $\frac{dF(\theta,x)}{dx} \in \mathbb{R}^{m \times m}$.

Given the IFT and necessary notation, we can now proceed to use the IFT to compute derivatives of the output of an iterative method, as depicted in Fig. 1. In particular, we focus on computing derivatives of the solution to a constrained optimization problem. We will use the optimality criteria of the optimization problem to define a system of equations, then apply the IFT to compute the Jacobian of the solution wrt the parameters. This methodology allows us to use the solution to an optimization problem as the output of a layer within a neural network, as done in OptNet [Amos and Kolter, 2017].

4 Differentiating through Optimization

As an introductory example, we will replace softmax with an equivalent function defined as the output of an optimization problem, then derive derivatives using the IFT. We will start by reviewing softmax and its expression as an optimization problem. After checking the conditions of the IFT hold, we can then compute derivatives. Since the Jacobian of softmax is known, we can directly verify that the IFT gives the correct answer.

4.1 Softmax

Softmax is often used to parameterize categorical distributions within neural networks, such as in attention layers. Given n items with independent utilities, where $\theta \in \mathbb{R}^n$ indicate preferences, the output of softmax gives the following distribution over items: $z_i = \mathrm{softmax}(\theta)_i = \frac{\exp(\theta_i)}{\sum_j \exp(\theta_j)}$, with $z \in [0, 1]^n$. While there is a closed-form equation for both the output of softmax as well as its Jacobian, we use it as an introduction to the mechanism behind OptNet (and other differentiable optimization layers) [Amos and Kolter, 2017, Agrawal et al., 2019].

The output of softmax is also the solution of the following constrained optimization problem [Gao and Pavel, 2018]:

$$\begin{aligned} \text{maximize} \quad & z^\top \theta + H(z) \\ \text{subject to} \quad & z^\top \mathbf{1} = 1 \\ & z_i \geq 0, \forall i, \end{aligned} \tag{2}$$

where $H(z) = -\sum_i z_i \log z_i$ is the entropy. The first term in the objective, $z^\top \theta$, is highest when z points in the same direction as θ. Given the constraint that z must sum to one and have nonnegative entries, maximizing just the first term subject to those constraints results in a solution that picks out the highest component of θ, i.e. $\mathrm{argmax}(\theta)$. The addition of the entropy term penalizes solutions

that put too much mass on just a few items, resulting in a smoothed version of argmax. We will refer to this entropy-regularized version as the softmax problem.[6]

Our goal is to compute the Jacobian of softmax $\frac{dz}{d\theta} = \frac{d\mathrm{softmax}(\theta)}{d\theta}$ using the IFT and the optimization problem above. While this may seem trivial because softmax has a closed form expression for both the output and Jacobian, it is a worthwhile exercise in applying the IFT. Applying the IFT to optimization problems consists of four steps:

1. Find a solution to the optimization problem.
2. Write down a system of equations derived from the optimality conditions.
3. Check that the conditions of the IFT hold.
4. Compute the derivative of the implicit solution mapping wrt the parameters.

We assume the first step has been done for us, and we have a solution z to the softmax problem.[7] We will then use the IFT to compute gradients of z wrt the parameters θ by following the rest of the steps.

Step 2: The KKT conditions determine the system of equations

In order to apply the IFT, we need a system of equations for which our outputs of interest are solution points for. The goal of this step is to choose a suitable system of equations. This differs from the example of the unit-circle, which was already defined by a system of equations.[8]

For solutions of optimization problems, the Karush-Kuhn-Tucker (KKT) conditions are a natural choice for defining such a system of equations. Given an optimization problem, the KKT conditions determine a system of equations that the solution must satisfy based on the optimality criteria [Karush, 1939, Kuhn and Tucker, 1951]. They are stationarity (the gradient should be 0 at a local optimum) and feasibility (the constraints of the problem should not be violated). For a thorough introduction to the KKT conditions, see chapter 5 of Boyd and Vandenberghe [2004] or the Wikipedia article

We will use the KKT conditions of the softmax problem in Eqn. 2 to determine the (vector-valued) function $F : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^m$ in the IFT. First, we introduce dual variables $u \in \mathbb{R}, v \in \mathbb{R}^n$ and write out the Lagrangian:

$$\mathcal{L}(\theta, z, u, v) = z^\top \theta + H(z) + u(z^\top \mathbf{1}_n - 1) + v^\top z.$$

[6]Removing the entropy regularization term results in the argmax optimization problem: maximize $z^\top \theta$, subject to $z^\top \mathbf{1}_n = 1$ and $z \succeq 0$. This is known as the support function of the set $\{z \mid z^\top \mathbf{1}_n = 1, z \succeq 0\}$.

[7]This is trivial for softmax since we can compute it using the closed form expression. However, in more general optimization problems, we would obtain z from a solver with a large execution trace.

[8]For a given solution point there may be many suitable systems, all of which would be viable as long as they satisfy the conditions of the IFT that we will check in step 3.

We therefore have the solution point (θ, z, u, v), with parameters θ and solution $x = (z, u, v)$. We then have the following necessary conditions for a solution (z, u, v), i.e. the KKT conditions:

$$\begin{aligned} \frac{d}{dz}\mathcal{L}(\theta, z, u, v) &= \mathbf{0}_n && \text{(stationarity)} \\ z^\top \mathbf{1}_n - 1 &= 0 && \text{(primal feasibility, equality)} \\ \operatorname{diag}(v) z &= \mathbf{0}_n && \text{(complementary slackness)} \\ z &\succeq \mathbf{0}_n && \text{(primal feasibility, inequality)} \\ v &\succeq \mathbf{0}_n && \text{(dual feasibility)} \end{aligned} \tag{3}$$

As we only need a system of equations with $2n + 1$ equations to determine the $2n + 1$ solution variables $x = (z, u, v)$, we use the first three conditions: stationarity, primal feasibility (equality), and complementary slackness.

In full, the system of equations $F(\theta, z, u, v) = 0$ we choose for the softmax problem is

$$\begin{aligned} \theta - \log(z) - \mathbf{1}_n + u\mathbf{1}_n + v &= \mathbf{0}_n \\ z^\top \mathbf{1}_n - 1 &= 0 \\ \operatorname{diag}(v) z &= \mathbf{0}_n. \end{aligned} \tag{4}$$

Note that the first and third equations are vector-valued.

This completes the second step, where we chose a subset of the KKT conditions in order to produce a nonlinear system of equations. We can now proceed to check the conditions of the IFT, which will determine whether F is locally well-behaved at a particular solution point (θ, z, u, v).

Step 3: Check that the IFT conditions hold at the solution point

Recall that the IFT requires the following three conditions, which must be checked on a case-by-case basis for particular solution points[9]:

- $F(\theta, z, u, v) = 0$,
- F has at least continuous first derivatives,
- $\det \frac{dF(\theta,z,u,v)}{d(z,u,v)} \neq 0$, or equivalently $\frac{dF(\theta,z,u,v)}{d(z,u,v)}$ is full rank.

Returning to the softmax problem, the first condition holds as we have a solution to the optimization problem and F was chosen using the KKT conditions.[10] The second condition also holds, as F has continuous first derivatives. All that remains is to check the third condition, that the Jacobian matrix $\frac{dF(\theta,z,u,v)}{d(z,u,v)}$ (evaluated at the solution point) is non-singular.

[9] If these conditions do not hold, the IFT does not apply at this particular combination of F and solution point. The derivative $\frac{dx}{d\theta}$ may still be computed via other means if it exists.

[10] Recall that softmax also has a closed form expression.

The Jacobian matrix $\frac{dF(\theta,z,u,v)}{d(z,u,v)} \in \mathbb{R}^{2n+1\times 2n+1}$ is given by

$$\frac{dF(\theta,z,u,v)}{d(z,u,v)} = \begin{bmatrix} -\mathrm{diag}(z)^{-1} & \mathbf{1}_n & I_{n\times n} \\ \mathbf{1}_n^\top & 0 & \mathbf{0}_n^\top \\ \mathrm{diag}(v) & \mathbf{0}_n & \mathrm{diag}(z) \end{bmatrix}. \tag{5}$$

Since a solution must be feasible, we know that $z^\top \mathbf{1}_n = 1$. However, the upper left block, $\mathrm{diag}(z)^{-1}$, contains a divide-by-zero term if any component $z_i = 0$.[11] To avoid this, we consider only strictly positive $z \succ 0$ for the IFT.[12] Strict positivity and complementary slackness imply $v = \mathbf{0}_n$, so the Jacobian reduces to a block upper-triangular matrix with lower-right block $\mathrm{diag}(z)$ and upper-left block

$$A = \begin{bmatrix} -\mathrm{diag}(z)^{-1} & \mathbf{1}_n \\ \mathbf{1}_n^\top & 0 \end{bmatrix}.$$

Since $\mathrm{diag}(z)$ is invertible and the Schur complement of A with respect to $-\mathrm{diag}(z)^{-1}$ is

$$0 - \mathbf{1}_n^\top \left(-\mathrm{diag}(z)\right) \mathbf{1}_n = z^\top \mathbf{1}_n = 1,$$

the block A is invertible as well. Therefore, the Jacobian of F is full rank and equivalently has nonzero determinant. This shows that the conditions of the IFT hold for the solution points that are feasible, optimal, and have strictly positive z.[13]

Step 4: Compute $\frac{dx}{d\theta}$

Now that we have a set of solution points where conditions of the IFT hold, we can use the IFT to compute $\frac{dz}{d\theta}$. Recall that we have the solution $x = (z, u, v)$; we will switch to x for brevity. The second part of the IFT (Eqn. 1) tells us that we can compute the Jacobian of the solution mapping $\frac{dx}{d\theta} = \frac{dx^*(\theta)}{d\theta} = -\left[\frac{dF(\theta,x)}{dx}\right]^{-1} \frac{dF(\theta,x)}{d\theta}$, then pick out the relevant components.

The second term, $\frac{dF(\theta,x)}{d\theta}$, is simple. Since θ only appears in the first vector-valued function of F (see Eqn. 4), we have

$$\frac{dF(\theta,x)}{d\theta} = \begin{bmatrix} I_{n\times n} \\ \mathbf{0}_{(n+1)\times n} \end{bmatrix}. \tag{6}$$

The large amount of sparsity allows us to skip some computation further down.

Next, we have to invert the Jacobian from Eqn. 5:

$$\left[\frac{dF(\theta,x)}{dx}\right]^{-1} = \begin{bmatrix} -\mathrm{diag}(z)^{-1} & \mathbf{1}_n & I_{n\times n} \\ \mathbf{1}_n^\top & 0 & 0 \\ \mathrm{diag}(v) & 0 & \mathrm{diag}(z) \end{bmatrix}^{-1}. \tag{7}$$

[11] This term was obtained by differentiating the entropy term of the Lagrangian, $H(z) = -\sum_i z_i \log z_i$. While we could use the convention $0 \log 0 = 0$, this does not fix the divide-by-zero issue with the second derivative, which we see here.

[12] We saw a similar issue in the unit circle example, where the derivative $\frac{dx}{d\theta}$ was undefined when $x = 0$ (see Fig. 2).

[13] Recall that the IFT applies at particular solution points, meaning we can pick and choose which points to analyze. In the case of softmax, we can perform a separate analysis for solution points on the boundary of the constraint set.

The remainder of this section is compute-intensive; feel free to skip ahead to Sec. 4.2 for a discussion on the limitations of applying the IFT. We use the block-wise inversion formula

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} (A - BD^{-1}C)^{-1} & 0 \\ 0 & (D - CA^{-1}B)^{-1} \end{bmatrix} \begin{bmatrix} I & -BD^{-1} \\ -CA^{-1} & I \end{bmatrix},$$

where

$$A = \begin{bmatrix} -\mathrm{diag}(z)^{-1} & \mathbf{1}_n \\ \mathbf{1}_n^\top & 0 \end{bmatrix} \qquad B = \begin{bmatrix} I_{n\times n} \\ 0 \end{bmatrix}$$
$$C = \begin{bmatrix} \mathrm{diag}(v) & 0 \end{bmatrix} \qquad D = \mathrm{diag}(z).$$

However, because $z \succ 0$, complementary slackness gives $v = 0$,[14] reducing the above to

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} A^{-1} & 0 \\ 0 & D^{-1} \end{bmatrix} \begin{bmatrix} I_{(n+1)\times(n+1)} & -BD^{-1} \\ 0 & I_{n\times n} \end{bmatrix}.$$

As we are interested in computing $\frac{dz}{d\theta}$, rather than the full derivative $\frac{dx}{d\theta}$ (recall $x = (z, u, v)$), in addition to the sparsity of $\frac{dF}{d\theta}$, we only have to solve for the upper-left $n \times n$ block of $A^{-1} \in \mathbb{R}^{n+1\times n+1}$. To do so, we will repeat the same block-wise inverse computation. Let us denote

$$A = \begin{bmatrix} -\mathrm{diag}(z)^{-1} & \mathbf{1}_n \\ \mathbf{1}_n^\top & 0 \end{bmatrix} = \begin{bmatrix} E & F \\ G & H \end{bmatrix}.$$

First, we compute the Schur complement of A,

$$A/E = H - GE^{-1}F = 0 - \mathbf{1}_n^\top(-\mathrm{diag}(z))\mathbf{1}_n = z^\top \mathbf{1}_n. \tag{8}$$

Since z is feasible, we have $A/E = 1$ due to the equality constraints (z must sum to 1 as a probability mass function). Then, we have

$$\begin{aligned} A^{-1} &= \begin{bmatrix} -\mathrm{diag}(z)^{-1} & \mathbf{1}_n \\ \mathbf{1}_n^\top & 0 \end{bmatrix}^{-1} \\ &= \begin{bmatrix} E & F \\ G & H \end{bmatrix}^{-1} \\ &= \begin{bmatrix} E^{-1} + E^{-1}F(A/E)^{-1}GE^{-1} & -E^{-1}F(A/E)^{-1} \\ -(A/E)^{-1}GE^{-1} & (A/E)^{-1} \end{bmatrix}. \end{aligned} \tag{9}$$

Plugging in,

$$\begin{aligned} A^{-1} &= \begin{bmatrix} -\mathrm{diag}(z) + \mathrm{diag}(z)\mathbf{1}_n\mathbf{1}_n^\top\mathrm{diag}(z) & \mathrm{diag}(z)\mathbf{1}_n \\ \mathbf{1}_n^\top\mathrm{diag}(z) & 1 \end{bmatrix} \\ &= \begin{bmatrix} -\mathrm{diag}(z) + zz^\top & z \\ z^\top & 1 \end{bmatrix}. \end{aligned} \tag{10}$$

[14] We apply the IFT to solutions where $z \succ 0$ due to a divide-by-zero issue in $\frac{\partial^2 H(z)}{\partial z_i^2} = -\frac{1}{z_i}$ for $H(z) = -\sum_j z_j \log z_j$.

Pulling out the top-left $n \times n$ block and negating it (see Eqns. 1 and 6) yields the Jacobian $\frac{dz}{d\theta} = \text{diag}(z) - zz^\top$, which agrees with directly differentiating softmax [Martins and Astudillo, 2016]. While we were able to take advantage of sparsity in this computation, such sparsity may not be available in general. Without sparsity, solving the linear system defined by the Jacobian of F, $\frac{dF(\theta,x)}{dx}$, costs $O(n^3)$ time and $O(n^2)$ space in this softmax example, since $x = (z, u, v)$ has dimension $2n + 1 = O(n)$. By contrast, computing softmax itself or applying its Jacobian to a vector costs $O(n)$, while explicitly materializing the full Jacobian $\text{diag}(z) - zz^\top$ costs $O(n^2)$ time and $O(n^2)$ space.

This concludes the exercise of using the IFT to differentiate through the softmax problem. While softmax itself has a closed form expression for both its output and derivative, we showed that the IFT can also be used to compute its derivative in a solver-agnostic manner. We started by expressing softmax as an optimization problem (Eqn. 2), then, assuming we had a solution to the softmax problem (regardless of whether it was obtained in closed-form or through an iterative method), reverse engineered a system of equations using the KKT conditions. We then restricted our attention to the set of solution points where we could successfully apply the IFT, then applied Eqn. 1 to compute the derivative at a solution point.

We can immediately apply the methodology developed in this section to more general optimization problems, as we will explore in Sec. 5. In those problems closed form solutions may not exist, and iterative solvers must be used where unrolling K solver steps and applying reverse-mode AD would take at least $\Omega(Kn)$ space when the iterates are n-dimensional, while naive implicit differentiation would instead store the dense Jacobian of the local system, taking $O(d^2)$ space for a d-dimensional system.

4.2 Limitations

In order to compute the derivative $\frac{dx}{d\theta}$, we had to invert the Jacobian of F (the system of equations derived from the KKT conditions), i.e. compute $\left[\frac{dF(\theta,x)}{dx}\right]^{-1}$. However, the first part of F (recall from Eqn. 4), the stationarity condition $\frac{d}{dz}\mathcal{L} = 0$ already involved derivatives of the Lagrangian $\mathcal{L}$ with respect to the primal variables. In general, this means that in order to apply the IFT to solutions of optimization problems, we must solve a linear system involving the Jacobian of the KKT conditions.[15] If this Jacobian/KKT matrix is dense and has dimension d, storing it costs $O(d^2)$ space and solving the corresponding linear system costs $O(d^3)$ time.[16] Thankfully, there are relatively cheap ways of approximating this computation, such as with approximate inverse-matrix-vector-product techniques [Rajeswaran et al., 2019, Lorraine et al., 2019a]. Lorraine et al. [2019a] in particular show that the derivative can be approximated in $O(n)$ space using a truncated Neumann series approximation for the inverse Hessian in the unconstrained setting.

Additionally, comparing the speed of computing derivatives via AD on a large trace versus the

[15] From the IFT, we can either work with the full inverse of this Jacobian or only the corresponding inverse-matrix-vector product, which can sometimes be cheaply approximated. In unconstrained problems this reduces to working with an inverse Hessian, while constrained problems lead to a larger KKT system that also includes dual variables.

[16] The number of solution variables scales with the number of primal variables, but also the number of constraints. This potentially makes applying the IFT to optimization problems with exponentially many constraints difficult.

IFT must be done on a case-by-case basis. For softmax, let us pretend we solved the softmax problem with an iterative method in K steps, each costing $O(n)$ time. Computing the derivative from the execution trace would then take $O(Kn)$ time using reverse-mode AD, while the naive IFT would take $O(n^3)$ time because the KKT system has size $2n + 1$. Under this simplified model, implicit differentiation is asymptotically faster only when $K > n^2$. In practice, the constant factors in the runtime will play a large factor in which approach is better, as well as any approximations made to make either approach more efficient[17].

4.3 Extensions

The methods we covered can be extended to variations of argmax problems other than the softmax problem.[18] The softmax problem altered the argmax problem by introducing entropy regularization. Rather than regularizing with entropy, one could instead alter the objective to find the Euclidean projection of the parameters onto the probability simplex, resulting in SparseMax [Martins and Astudillo, 2016]. While the output of softmax variants often have a closed form expression, the IFT provides another way of deriving their Jacobians and could potentially pave the way for differentiating through argmax problems that do not have closed-form expressions, such as projections onto the $(n, k)-$simplex, which can be interpreted as selecting k elements from n [Kong et al., 2020].

More generally, the IFT can be applied to cases where, unlike softmax, we do not have an explicit functional form (i.e., the unit circle), and outputs are governed only by a system of equations. This includes more general optimization problems, such as quadratic programs [Amos and Kolter, 2017] or other convex optimization problems [Agrawal et al., 2019].

5 OptNet

OptNet generalizes the methodology applied above to the softmax problem by extending the optimization problems considered, in particular including parameterized constraints. This allows us to learn not only the objective, but also the constraints. Explicitly incorporating families of constraints in models with optimization layers allows them to perform well on tasks with rigid constraints, such as learning to play Sudoku from only inputs and outputs [Amos and Kolter, 2017].

OptNet applies the IFT to quadratic programs (QPs) in particular. As the simplest nonlinear optimization problem, QPs strike a balance between expressivity and computational tractability [Frank and Wolfe, 1956]. The methodology remains the same as the softmax problem: Given a QP and a solution, use the KKT conditions to produce a system of equations, then apply the IFT to compute the derivative of the solution wrt the parameters of the objective and constraints.

[17] For unrolled AD, we could not fully solve the optimization problem and use the truncated trace. For the IFT, vector-inverse Hessian vector product approximations other than the Neumann series include conjugate gradient, identity approximations, and other factorizations such as K-fac [Martens and Grosse, 2015].

[18] The argmax problem is given by maximize $z^\top \theta$, subject to $z^\top \mathbf{1}_n = 1$ and $z \succeq 0$, otherwise known as the support function of the constraint set.

Quadratic programs take the following form:

$$\begin{aligned} \text{minimize} \quad & \frac{1}{2} z^\top Q z + q^\top z \\ \text{subject to} \quad & Gz \leq h \\ & Az = b, \end{aligned} \tag{11}$$

where we optimize over $z \in \mathbb{R}^n$ and the parameters are $\theta = \{Q, q, A, b, G, h\}$, with $Q \in \mathbb{R}^{n \times n} \succeq 0, q \in \mathbb{R}^n, A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m, G \in \mathbb{R}^{p \times n}, h \in \mathbb{R}^p$. Compared to the softmax problem in Eqn. 2, the main difference is the learnable parameters in the constraints. As the application of the IFT is very similar, we will not cover it in as much detail as the softmax problem.

Similar to the softmax problem, we will first assume we already have a solution. Unlike in the softmax problem, we assume the solution to the QP is provided by a solver, as there may not be a closed-form solution.[19] In this case, the execution trace of the solver may be too large to store in memory, necessitating the use of the IFT.

Step 2: The KKT conditions determine the system of equations

The Lagrangian of the QP is given by

$$\mathcal{L}(\theta, z, u, v) = \frac{1}{2} z^\top Q z + q^\top z + u^\top (Gz - h) + v^\top (Az - b),$$

introducing dual variables $u \in \mathbb{R}^p, v \in \mathbb{R}^m$, where we have the solution $x = (z, u, v)$.

We use the following subset of the KKT conditions to determine our system of equations: stationarity, primal feasibility (equality), and complementary slackness. This yields, written in full,

$$\begin{aligned} Qz + q + G^\top u + A^\top v &= \mathbf{0}_n \\ \operatorname{diag}(u)(Gz - h) &= \mathbf{0}_p \\ Az - b &= \mathbf{0}_m. \end{aligned} \tag{12}$$

Step 3: Check the conditions of the IFT

As with the softmax problem, the only nontrivial part is checking that the Jacobian matrix $\frac{dF(\theta,x)}{dx}$ is not singular at the solution point(s) of interest (θ, x). This computation is more involved for QPs than the softmax problem. See Theorem 1 and its proof in the OptNet paper for more details [Amos and Kolter, 2017].

Step 4: Compute $\frac{dx}{d\theta}$

We would then compute $\frac{dx}{d\theta} = -\left[\frac{dF(\theta,x)}{dx}\right]^{-1} \frac{dF(\theta,x)}{d\theta}$, matching Eqn. 1, by hand or numerically. Amos and Kolter [2017] also show how to obtain the vector-Jacobian product efficiently using quantities readily available from a QP solver.

[19] One contribution of OptNet was the extension of a state-of-the-art interior point solver [Amos and Kolter, 2017], and its adaptation to parallel machines (GPUs) and batch processing. While outside the scope of this report, see the paper by Amos and Kolter [2017] for the details.

6 Discussion

In this note, we walked through the application of the implicit function theorem to optimizing the parameters of an optimization problem, as a function of a solution. The process was quite involved, involving a large number of manual calculations. Ironically, this contrasts with the philosophy of **automatic** differentiation, which seeks to minimize user error by minimizing the number of manual calculations.[20] Work by Blondel et al. [2021] seeks to address this via applying automatic differentiation to the system of equations central to the application of the IFT to bilevel optimization. We are excited to see where these methods find applications!

Acknowledgements

We thank Yuntian Deng, Daniel Fried, Woojeong Kim, Jack Morris, and Alexander Rush for helpful feedback.

References

A. Agrawal, B. Amos, S. T. Barratt, S. P. Boyd, S. Diamond, and J. Z. Kolter. Differentiable convex optimization layers. *CoRR*, abs/1910.12430, 2019. URL `http://arxiv.org/abs/1910.12430`.

B. Amos and J. Z. Kolter. Optnet: Differentiable optimization as a layer in neural networks. *CoRR*, abs/1703.00443, 2017. URL `http://arxiv.org/abs/1703.00443`.

Q. Bertrand, Q. Klopfenstein, M. Blondel, S. Vaiter, A. Gramfort, and J. Salmon. Implicit differentiation of lasso-type models for hyperparameter optimization, 2020.

M. Blondel, Q. Berthet, M. Cuturi, R. Frostig, S. Hoyer, F. Llinares-López, F. Pedregosa, and J. Vert. Efficient and modular implicit differentiation. *CoRR*, abs/2105.15183, 2021. URL `https://arxiv.org/abs/2105.15183`.

S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, March 2004. ISBN 0521833787. URL `http://www.amazon.com/exec/obidos/redirect?tag=citeulike-20&path=ASIN/0521833787`.

C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. *CoRR*, abs/1703.03400, 2017. URL `http://arxiv.org/abs/1703.03400`.

M. Frank and P. Wolfe. An algorithm for quadratic programming. *Naval Research Logistics Quarterly*, 3(1-2):95–110, 1956. URL `https://EconPapers.repec.org/RePEc:wly:navlog:v:3:y:1956:i:1-2:p:95-110`.

[20] The degree of manual calculation in applying the IFT is unsurprising, as it generalizes forward and reverse-mode automatic differentiation. Griewank and Walther [2008] show that we can derive forward and reverse-mode automatic differentiation from the IFT, through Schur-complements as we showed for the particular case of softmax.

B. Gao and L. Pavel. On the properties of the softmax function with application in game theory and reinforcement learning, 2018.

A. N. Gomez, M. Ren, R. Urtasun, and R. B. Grosse. The reversible residual network: Backpropagation without storing activations. *CoRR*, abs/1707.04585, 2017. URL http://arxiv.org/abs/1707.04585.

S. Gould, R. Hartley, and D. Campbell. Deep declarative networks: A new hope. *CoRR*, abs/1909.04866, 2019. URL http://arxiv.org/abs/1909.04866.

A. Griewank and A. Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. SIAM, Philadelphia, 2nd edition, 2008.

W. Karush. *Minima of functions of several variables with inequalities as side conditions.* PhD thesis, Thesis (S.M.)–University of Chicago, Department of Mathematics, December 1939., 1939.

Y. Kim, S. Wiseman, A. C. Miller, D. Sontag, and A. M. Rush. Semi-amortized variational autoencoders, 2018.

W. Kong, W. Krichene, N. Mayoraz, S. Rendle, and L. Zhang. Rankmax: An adaptive projection alternative to the softmax function. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 633–643. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper/2020/file/070dbb6024b5ef93784428afc71f2146-Paper.pdf.

S. Krantz and H. Parks. *The Implicit Function Theorem: History, Theory, and Applications*. Birkhäuser, 01 2003. ISBN 978-1-4612-6593-1. doi: 10.1007/978-1-4612-0059-8.

H. W. Kuhn and A. W. Tucker. Nonlinear programming. In *Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability, 1950*, pages 481–492, Berkeley and Los Angeles, 1951. University of California Press.

J. Lorraine, P. Vicol, and D. Duvenaud. Optimizing millions of hyperparameters by implicit differentiation. *CoRR*, abs/1911.02590, 2019a. URL http://arxiv.org/abs/1911.02590.

J. Lorraine, P. Vicol, and D. Duvenaud. Optimizing millions of hyperparameters by implicit differentiation. *CoRR*, abs/1911.02590, 2019b. URL http://arxiv.org/abs/1911.02590.

D. Maclaurin, D. Duvenaud, and R. P. Adams. Gradient-based hyperparameter optimization through reversible learning, 2015.

J. Martens and R. B. Grosse. Optimizing neural networks with kronecker-factored approximate curvature. *CoRR*, abs/1503.05671, 2015. URL http://arxiv.org/abs/1503.05671.

A. F. T. Martins and R. F. Astudillo. From softmax to sparsemax: A sparse model of attention and multi-label classification. *CoRR*, abs/1602.02068, 2016. URL http://arxiv.org/abs/1602.02068.

A. Rajeswaran, C. Finn, S. M. Kakade, and S. Levine. Meta-learning with implicit gradients. *CoRR*, abs/1909.04630, 2019. URL http://arxiv.org/abs/1909.04630.

S. Venkataraman and B. Amos. Neural fixed-point acceleration for convex optimization. *CoRR*, abs/2107.10254, 2021. URL https://arxiv.org/abs/2107.10254.